

A football decision-support agent that estimates the probability of another goal, compares it with live market pricing, and records human-approved paper trades.

Overview

GoalEdge ingests live TxLINE fixture, score and odds data, reconstructs the current match state, runs a trained logistic-regression model, and compares the model probability with the market-implied probability.

It never executes real-money bets. A qualifying opportunity must pass model, market, freshness and fixture-state checks before a user can approve a **paper trade**.

Architecture

TxLINE API → server-side authentication and normalisation → GET /api/txline/snapshot → 5-second monitoring engine → GoalHistoryTracker → trained model → model-vs-market decision → human-approved paper trade

TxLINE credentials remain server-side. The browser receives only normalised fixture, score, odds and provider-timestamp data.

The polling engine is created, stored and started inside the same React useEffect. This fixes the earlier effect-ordering issue where the engine existed but never began polling. A successful request changes the badge from **Connecting to TxLINE...** to **Live TxLINE connected**.

Model and calculations

```
ml/models/next_goal_none_logistic_v1.json
lib/model/nextGoalNoneModel.ts
```

The model directly predicts **P(no further goal)**. GoalEdge derives the user-facing outcome:

```
P(another goal) = 1 - P(no further goal)
Market probability = 1 / decimal odds
Edge = model probability - market probability
Fair odds = 1 / model probability
```

The fixed feature order is: minute, minute squared, home score, away score, total goals, goal difference, draw state, time since last goal, home red cards and away red cards.

The exported scaler, coefficients, intercept and feature order are validated at runtime. Missing or ambiguous live inputs make the result unavailable rather than triggering a fabricated fallback.

Integrity and safeguards

- No synthetic live fixtures, odds or probabilities.
- API tokens are never exposed to the browser.
- Future match events are never used as model inputs.
- Finished matches cannot generate new BUY signals.
- Market data must be fresh and the selection must be genuinely published by TxLINE.
- A trained-model BUY and human approval are both required.
- Execution is paper-trade only; no real money is placed.
- Malformed or legacy browser records are filtered from trade history.

Approved paper trades retain the fixture, selection, model probability, market probability, edge, odds and market timestamp for auditability.

Historical replay

Downloaded TxLINE replay data is read from `ml/data/raw/{fixture_id}/`. Raw files are intentionally gitignored and are not required for the normal app to build or run.

Historical reconstruction uses only events available at or before the selected minute, preventing future-information leakage. When proprietary replay files are unavailable, clearly separated bundled historical demonstration data can keep the demo functional.

```
.\ml\.venv\Scripts\python.exe ml\download_replay.py
--list-eligible-fixtures
```

Local setup and verification

```
git clone https://github.com/HW006-J/world-cup-trading-agent.git
cd world-cup-trading-agent
npm ci
Copy-Item .env.example .env.local
npm run dev -- --hostname 127.0.0.1
```

Required environment-variable names: `TXLINE_API_TOKEN` and `TXLINE_BASE_URL`. Secret values must remain in `.env.local`.

```
npx tsc --noEmit
npm run lint
npm test
npm run build
npm run diagnose:txline
```

Current limitations

Live fixture availability depends on the TxLINE environment and match schedule. Raw historical replays must be downloaded locally. Paper trades are stored in the current browser rather than a shared database. GoalEdge is a focused decision-support prototype for a specific football market, not a general betting system.